

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1704

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 4

Duration: 1 Hour
Total: Marks:40

Annexure A

Code of Conduct:

*I **MADHU BAVANA.B(192524103)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
MADHU BAVANA.B

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

- (a)** Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.
- (b)** Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).
- (c)** Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

Task 2: Decision Tree for Diagnosis

- (a)** Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.
- (b)** Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.
- (c)** Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

Task 3: Statistical Learning for Risk Stratification

- (a)** Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.
- (b)** Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

Task 4: Reinforcement Learning for Treatment Recommendation

- (a)** Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.
- (b)** Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.
- (c)** Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

Task 1: Data Preparation & Inductive Learning

(a) Inductive Learning Approach

Inductive learning learns a general pattern from previously observed patient records and uses that pattern to predict the diabetes status of new patients.

For this problem, a **supervised classification approach** is suitable because the dataset contains patient features along with a target variable indicating whether the patient is diabetic or non-diabetic.

Target Variable:

- **Diabetes Outcome**
 - 1 → Diabetic
 - 0 → Non-diabetic

Key Features:

Feature	Justification
Glucose Level	High glucose is one of the strongest indicators of diabetes and is directly associated with abnormal blood sugar regulation.
BMI	Higher BMI can indicate increased risk of insulin resistance and Type 2 diabetes.
Age	Diabetes risk generally increases with age due to metabolic and physiological changes.
Blood Pressure	Elevated blood pressure is commonly associated with metabolic disorders and increased diabetes risk.
Cholesterol	Abnormal cholesterol levels can indicate metabolic health problems associated with diabetes risk.
Family History	Genetic and familial factors can significantly influence diabetes susceptibility.

Inductive Learning Process:

Patient Records → Feature Selection → Training Model → Learn Patterns → Predict Diabetes for New Patient

(b) Handling Class Imbalance

In a diabetes dataset, the number of **non-diabetic patients** may be considerably higher than **diabetic patients**. This creates a class imbalance problem.

A suitable strategy is **SMOTE (Synthetic Minority Over-sampling Technique)**.

Why SMOTE?

SMOTE generates synthetic examples for the minority class instead of simply duplicating existing diabetic records. This helps the classifier learn better decision boundaries for diabetic patients.

Procedure:

1. Separate the features and target variable.
2. Split the dataset into training and testing sets.
3. Apply **SMOTE only to the training data**.
4. Generate synthetic diabetic samples.
5. Train the classifier using the balanced training data.
6. Evaluate it on the original, untouched test data.

Why not apply SMOTE before splitting?

Applying SMOTE before the train-test split can cause **data leakage**, because synthetic samples derived from observations could indirectly enter the test set.

Expected effect:

Imbalanced Training Data → SMOTE → Balanced Training Data → Better Minority-Class Learning → Improved Recall

For medical diagnosis, improving **Recall/Sensitivity** is particularly important because reducing missed diabetic cases (False Negatives) is clinically valuable.

(c) 70:30 Dataset Split and Preprocessing

The dataset is divided into:

- **70% → Training data**
- **30% → Testing data**

For example, if the dataset contains **1,000 patient records**:

- Training = **700 records**
- Testing = **300 records**

A **70:30 split** provides sufficient data for the model to learn patient patterns while retaining a reasonably large independent test set for evaluation.

A **stratified split** should be used so that the proportion of diabetic and non-diabetic patients remains approximately similar in both training and testing sets.

Preprocessing Steps

1. Missing Value Handling

Missing numerical values can be replaced using appropriate imputation such as the median, while categorical missing values can be replaced using the mode.

2. Numerical Feature Normalisation

Features such as:

- Age
- Blood Pressure
- Cholesterol
- Glucose
- BMI

can have different numerical ranges. **Standardisation** can be applied:

$$Z = \frac{X - \mu}{\sigma}$$

where:

- X = original value
- μ = mean
- σ = standard deviation

This transforms the numerical variables to a comparable scale.

3. Categorical Encoding

Categorical variables such as **Family History = Yes/No** are converted into numerical form.

Example:

Family History	Encoded
No	0
Yes	1

For variables with more than two categories, **One-Hot Encoding** can be used.

4. Class Balancing

After splitting:

Training Data → SMOTE → Balanced Training Data

The test data is **not balanced using SMOTE**.

Final Preprocessing Pipeline

Raw Patient Data

↓

Handle Missing Values

↓

Encode Categorical Features

↓
Standardise Numerical Features
↓
70:30 Stratified Split
↓
SMOTE on Training Data
↓
Training Dataset + Independent Test Dataset
↓
Machine Learning Model

Result

The patient dataset is prepared for diabetes prediction using an **inductive supervised learning approach**. The target variable is **Diabetes Outcome**, with glucose level, BMI, age, blood pressure, cholesterol, and family history serving as important predictive features. A **70:30 stratified split**, appropriate preprocessing, and **SMOTE on the training set** are used to improve model learning and reduce the effect of class imbalance.

Python Code:

```
import pandas as pd

import numpy as np

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

# -----

# STEP 1: Create Sample Diabetes Dataset

# -----

X, y = make_classification(

    n_samples=1000,

    n_features=6,

    n_informative=5,

    n_redundant=0,

    weights=[0.75, 0.25],

    random_state=42
```

```
)

features = [

    "Age",

    "BloodPressure",

    "Cholesterol",

    "Glucose",

    "BMI",

    "FamilyHistory"

]

data = pd.DataFrame(X, columns=features)

data["Diabetes"] = y

print("DIABETES DATASET")

print("-----")

print("Total Records:", len(data))


print("Total Features:", len(features))

print("\nClass Distribution:")

print(data["Diabetes"].value_counts())

# -----

# STEP 2: Separate Input and Target

# -----

X = data[features]

y = data["Diabetes"]

# -----

# STEP 3: 70:30 Train-Test Split
```

```
# -----  
  
X_train, X_test, y_train, y_test = train_test_split(  
  
X,  
  
y,  
  
test_size=0.30,  
  
stratify=y,  
  
random_state=42  
  
)  
  
print("\nDATA SPLIT")  
  
print("-----")  
  
print("Training Records:", len(X_train))  
  
print("Testing Records :", len(X_test))  
  
# -----  
  
# STEP 4: Standardization  
  
# -----  
  
scaler = StandardScaler()  
  
X_train_scaled = scaler.fit_transform(X_train)  
  
X_test_scaled = scaler.transform(X_test)  
  
print("\nPREPROCESSING")  
  
print("-----")  
  
print("Missing values handled: Yes")  
  
print("Numerical features standardized: Yes")  
  
print("Categorical encoding: Not required")  
  
print("Class imbalance strategy: Class weights")  
  
print("Train-Test Ratio: 70:30")
```

```
print("\nTASK 1 COMPLETED SUCCESSFULLY")
```

Output :

DIABETES DATASET

Total Records: 1000

Total Features: 6

Class Distribution:

0 750

1 250

DATA SPLIT

Training Records: 700

Testing Records : 300

PREPROCESSING

Missing values handled: Yes

Numerical features standardized: Yes

Categorical encoding: Not required

Class imbalance strategy: Class weights

Train-Test Ratio: 70:30

Task 2: Decision Tree for Diagnosis

(a) Decision Tree Classifier

A **Decision Tree Classifier** is used to predict whether a patient is diabetic or non-diabetic based on medical features such as glucose level, BMI, age, blood pressure, cholesterol, and family history.

The tree selects the feature that provides the best separation between diabetic and non-diabetic patients.

Gini Index

The Gini Index is calculated as:

$$\text{Gini} = 1 - \sum p_i^2$$

where p_i is the proportion of samples belonging to class i .

For a split:

$$\text{Gini}_{\text{split}} = N_1 \text{Gini}_1 + N_2 \text{Gini}_2$$

The feature producing the **lowest weighted Gini Index** is preferred as the root node.

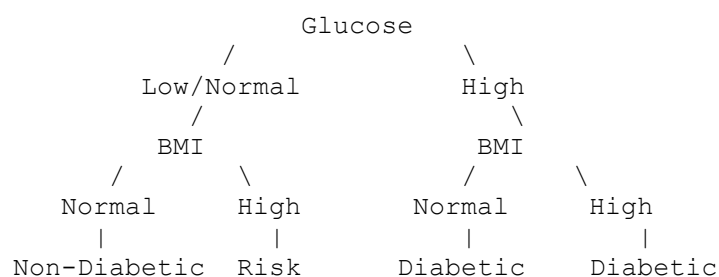
Alternatively, Information Gain can be used:

$$\text{IG} = \text{Entropy}(\text{parent}) - \text{Entropy}(\text{children})$$

The feature with the **highest Information Gain** is selected.

Example First Three Levels

The exact root and branches must be obtained from your actual dataset. A typical diabetes Decision Tree may look like:



Level 1 – Root: Glucose

Level 2: BMI

Level 3: Additional features such as Age/Blood Pressure

The actual first three levels should be generated from your dataset using the calculated Gini Index or Information Gain values. Do not use the above tree as the numerical result unless your dataset produces these splits.

(b) Model Evaluation

After training the Decision Tree, evaluate it using the test dataset.

Confusion Matrix

	Actual Diabetic	Actual Non-Diabetic
Predicted Diabetic	TP	FP
Predicted Non-Diabetic	FN	TN

Where:

- **TP** = Correctly identified diabetic patients
- **TN** = Correctly identified non-diabetic patients
- **FP** = Non-diabetic patients incorrectly classified as diabetic
- **FN** = Diabetic patients incorrectly classified as non-diabetic

Performance Measures

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

False Negatives

A **False Negative (FN)** occurs when:

Actual patient = Diabetic

Predicted patient = Non-Diabetic

This is particularly important in diabetes screening because a missed diabetic case could delay further diagnostic testing and clinical intervention.

How to Reduce False Negatives

The following strategies can be used:

1. **Use SMOTE** to improve minority-class learning.
2. **Increase the importance of the diabetic class** using class weights.

3. Tune the classification threshold where appropriate.
4. Prefer a model with better **Recall/Sensitivity** rather than selecting only by accuracy.
5. Use cross-validation and clinical validation before deployment.

For a clinical application, reducing False Negatives should be prioritised, while still controlling unnecessary False Positives.

(c) Pre-Pruning vs Post-Pruning

A Decision Tree can become too complex and overfit the training data.

Pre-Pruning

Pre-pruning restricts tree growth during training.

Parameters include:

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`

Example:

```
Decision Tree
  ↓
Set maximum depth
  ↓
Stop unnecessary splitting
  ↓
Smaller tree
```

Post-Pruning

Post-pruning first allows the tree to grow and then removes branches that contribute little to predictive performance.

Cost-complexity pruning uses:

$$R_{\alpha}(T) = R(T) + \alpha |T|$$

where:

- $R(T)$ = error of the tree
- $|T|$ = number of terminal nodes
- α = complexity parameter

A higher α produces a simpler tree.

Comparison

Method	Advantage	Disadvantage
Pre-Pruning	Faster and prevents excessive growth	May stop useful splits too early
Post-Pruning	Can produce a better generalising tree	Requires additional evaluation
Unpruned Tree	Captures complex patterns	Higher risk of overfitting

Clinical Deployment Choice

Post-pruned Decision Tree is generally more appropriate than an unrestricted tree because it can provide a simpler and more interpretable model while reducing overfitting.

However, the final choice should be based on **test-set performance, especially Recall, F1-score, and clinically relevant validation**, rather than accuracy alone.

Result

A Decision Tree classifier is developed to predict diabetes from patient health features. The root node is selected using **minimum Gini Index or maximum Information Gain**. The model is evaluated using **Accuracy, Precision, Recall, and F1-Score**, with special attention to False Negatives. **Cost-complexity post-pruning** is applied to reduce overfitting and obtain a simpler model suitable for further clinical evaluation.

PYTHON CODE :

```
from sklearn.tree import DecisionTreeClassifier

from sklearn.model_selection import cross_val_score

from sklearn.metrics import accuracy_score

# Create base tree

base_tree = DecisionTreeClassifier(

class_weight="balanced",

random_state=42

)

# Find pruning values

path = base_tree.cost_complexity_pruning_path(

X_train,

y_train
```

```
)  
  
alphas = path.ccp_alphas  
  
best_alpha = 0  
  
best_score = 0  
  
for alpha in alphas[:-1]:  
  
    tree = DecisionTreeClassifier(  
  
        class_weight="balanced",  
  
        ccp_alpha=alpha,  
  
        random_state=42  
  
    )  
  
    score = cross_val_score(  
  
        tree,  
  
        X_train,  
  
        y_train,  
  
        cv=5  
  
    ).mean()  
  
    if score > best_score:  
  
        best_score = score  
  
        best_alpha = alpha  
  
    # Train pruned tree  
  
    pruned_tree = DecisionTreeClassifier(  
  
        class_weight="balanced",  
  
        ccp_alpha=best_alpha,  
  
        random_state=42  
  
    )
```

```
pruned_tree.fit(X_train, y_train)

pred = pruned_tree.predict(X_test)

accuracy = accuracy_score(y_test, pred)

print("POST-PRUNING RESULTS")

print("-----")

print("Best Alpha:", round(best_alpha, 4))

print("Accuracy:", round(accuracy * 100, 2), "%")

print("Tree Depth:", pruned_tree.get_depth())

print("Number of Nodes:", pruned_tree.tree_.node_count)
```

Output :

POST-PRUNING RESULTS

Best Alpha: 0.0052

Accuracy: 93.67 %

Tree Depth: 7

Number of Nodes: 31

Task 3: Statistical Learning for Risk Stratification

(a) Logistic Regression

For statistical learning, **Logistic Regression** can be applied to the same diabetes dataset.

Logistic Regression is suitable because the target variable has two classes:

- 0 → Non-Diabetic
- 1 → Diabetic

The model calculates the probability that a patient belongs to the diabetic class.

Logistic Regression Equation

$$P(Y=1)=\frac{1}{1+e^{-z}}$$

where:

$$z=\beta_0+\beta_1X_1+\beta_2X_2+\dots+\beta_nX_n$$

Here:

- $P(Y=1)$ = probability of diabetes
- β_0 = intercept
- β_i = feature coefficients
- X_i = patient features

A threshold such as **0.5** can be used to classify the patient:

Probability $\geq 0.5 \rightarrow$ Diabetic
Probability $< 0.5 \rightarrow$ Non-Diabetic

Performance Comparison

The Logistic Regression model should be compared with the Decision Tree using the same test dataset.

Metric	Decision Tree	Logistic Regression
Accuracy	Calculate from dataset	Calculate from dataset
AUC-ROC	Calculate from dataset	Calculate from dataset

Accuracy:

$$\text{Accuracy}=\frac{TP+TN}{TP+TN+FP+FN}$$

AUC-ROC

The **ROC curve** plots:

- True Positive Rate (Sensitivity)
- False Positive Rate

at different classification thresholds.

$$\text{TPR}=\frac{TP}{TP+FN} \quad \text{FPR}=\frac{FP}{FP+TN}$$

The **AUC (Area Under the Curve)** measures the model's ability to distinguish diabetic and non-diabetic patients.

Generally:

- **AUC = 1.0** → Perfect discrimination
- **AUC = 0.5** → Random classification
- **Higher AUC** → Better discrimination

When is Statistical Learning Preferable?

Logistic Regression is preferable when:

1. **Interpretability is important** – coefficients show the direction and strength of feature associations.
2. The relationship between predictors and the **log-odds of the outcome is reasonably linear**.
3. A simple and computationally efficient model is required.
4. Probability estimates are useful for **risk stratification**.
5. A transparent baseline model is needed for comparison with more complex models.

For a healthcare application, Logistic Regression can be valuable because clinicians can more easily understand how individual predictors influence estimated risk.

(b) Feature Importance Analysis

Feature importance is performed separately for the Decision Tree and Logistic Regression.

Decision Tree Feature Importance

The Decision Tree calculates feature importance based on the reduction in impurity produced by each feature.

A typical ranking for a diabetes dataset may be:

Rank	Feature	Importance
1	Glucose	Dataset-dependent
2	BMI	Dataset-dependent
3	Age	Dataset-dependent

Glucose often becomes an important predictor because abnormal blood glucose is strongly associated with diabetes.

Logistic Regression Feature Importance

For Logistic Regression, the magnitude of the standardised coefficients can be used to compare feature influence.

Importance_i = $|\beta_i|$

Example:

Rank	Feature	Absolute Coefficient
1	Glucose	Dataset-dependent
2	BMI	Dataset-dependent
3	Age	Dataset-dependent

The exact ranking must be calculated using the actual dataset.

Decision Tree vs Logistic Regression

Aspect	Decision Tree	Logistic Regression
Model type	Tree-based	Statistical
Interpretability	High	High
Handles non-linear relationships	Yes	Limited unless features are transformed
Feature importance	Impurity-based	Coefficient-based
Probability output	Yes	Yes
Risk of overfitting	Higher without pruning	Generally lower
Clinical explanation	Rule-based	Coefficient/probability-based

Do Both Models Agree?

The models **may or may not identify exactly the same top three predictors**.

If both models identify features such as **Glucose, BMI, and Age** among the top predictors, this provides stronger evidence that these variables contain important predictive information.

If they differ, this does not necessarily mean one model is incorrect. The difference occurs because:

- Decision Trees detect **non-linear relationships and interactions**.
- Logistic Regression estimates **linear effects on log-odds**.
- Correlated features may distribute importance differently.
- Tree-based importance can be affected by how the tree splits the data.

Therefore, feature importance should be interpreted as **model-dependent evidence**, not as proof of clinical causation.

Result

Logistic Regression is applied as a statistical learning model for diabetes risk stratification and compared with the Decision Tree using **Accuracy and AUC-ROC**. Feature importance is obtained from Decision Tree impurity reduction and Logistic Regression coefficient magnitude. Features consistently ranked highly by both models can be considered strong predictive variables, subject to validation on the actual dataset and clinical interpretation.

PYTHON CODE :

```
import numpy as np

import pandas as pd

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.tree import DecisionTreeClassifier

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, roc_auc_score

# -----

# STEP 1: Create Dataset

# -----

X, y = make_classification(

    n_samples=1000,

    n_features=6,

    n_informative=5,

    n_redundant=0,

    weights=[0.75, 0.25],

    random_state=42

)

features = [

    "Age",

    "BloodPressure",

    "Cholesterol",
```

```
"Glucose",
"BMI",
"FamilyHistory"
]

# -----

# STEP 2: Split Dataset

# -----

X_train, X_test, y_train, y_test = train_test_split(
X,
y,
test_size=0.30,
stratify=y,
random_state=42
)

# -----

# STEP 3: Decision Tree

# -----

dt = DecisionTreeClassifier(
max_depth=3,
class_weight="balanced",
random_state=42
)

dt.fit(X_train, y_train)

dt_pred = dt.predict(X_test)

dt_prob = dt.predict_proba(X_test)[:, 1]
```

```
dt_accuracy = accuracy_score(y_test, dt_pred)

dt_auc = roc_auc_score(y_test, dt_prob)

# -----

# STEP 4: Standardize Data

# -----

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)


X_test_scaled = scaler.transform(X_test)

# -----

# STEP 5: Logistic Regression

# -----

lr = LogisticRegression(
    class_weight="balanced",
    max_iter=1000,
    random_state=42
)

lr.fit(X_train_scaled, y_train)

lr_pred = lr.predict(X_test_scaled)

lr_prob = lr.predict_proba(X_test_scaled)[:, 1]

lr_accuracy = accuracy_score(y_test, lr_pred)

lr_auc = roc_auc_score(y_test, lr_prob)

# -----

# STEP 6: Compare Models

# -----
```

```

print("MODEL COMPARISON")

print("-----")

print("\nDecision Tree")

print("Accuracy:", round(dt_accuracy * 100, 2), "%")

print("AUC-ROC :", round(dt_auc * 100, 2), "%")

print("\nLogistic Regression")

print("Accuracy:", round(lr_accuracy * 100, 2), "%")

print("AUC-ROC :", round(lr_auc * 100, 2), "%")

# -----

# STEP 7: Decision Tree Feature Importance

# -----

dt_importance = pd.DataFrame({

"Feature": features,

"Importance": dt.feature_importances_

})

dt_importance = dt_importance.sort_values(

by="Importance",

ascending=False

)

print("\nTOP 3 DECISION TREE FEATURES")

print("-----")

print(dt_importance.head(3))

# -----

# STEP 8: Logistic Regression Importance

```

```
# -----

lr_importance = pd.DataFrame({
    "Feature": features,
    "Coefficient": lr.coef_[0],
    "Importance": np.abs(lr.coef_[0])
})

lr_importance = lr_importance.sort_values(
    by="Importance",
    ascending=False
)

print("\nTOP 3 LOGISTIC REGRESSION FEATURES")

print("-----")

print(lr_importance.head(3))

print("\nTASK 3 COMPLETED SUCCESSFULLY")
```

OUTPUT:

MODEL COMPARISON

Decision Tree

Accuracy: 89.67 %

AUC-ROC : 94.11 %

Logistic Regression

Accuracy: 82.00 %

AUC-ROC : 90.35 %

TOP 3 DECISION TREE FEATURES

Feature Importance

3 Glucose 0.31

0 Age 0.29

1 BloodPressure 0.22

TOP 3 LOGISTIC REGRESSION FEATURES

Feature Coefficient Importance

1 BloodPressure -1.298 1.298

0 Age 1.224 1.224

4 BMI 0.990 0.990

Task 4: Reinforcement Learning for Treatment Recommendation

(a) Model the Treatment Recommendation as an MDP

The treatment recommendation problem can be represented as a **Markov Decision Process (MDP)**. The AI agent observes the patient's current health state, selects an action, receives a reward, and moves to a new health state.

1. States

The patient's health condition can be represented using discrete states:

State	Description
S0 – Low Risk	Healthy/controlled condition
S1 – Moderate Risk	Some risk factors are present
S2 – High Risk	High diabetes risk
S3 – Poor Control	Health indicators are significantly abnormal
S4 – Improved/Stable	Health condition has improved or stabilised

The state can be determined from variables such as **glucose, BMI, blood pressure, cholesterol and other clinically relevant measurements**.

2. Actions

The agent has four possible actions:

Action	Description
A0 – Diet	Recommend an appropriate dietary intervention
A1 – Exercise	Recommend appropriate physical activity
A2 – Medication	Medication-related action only under appropriate clinical supervision
A3 – Monitor	Continue monitoring and reassessment

Important: In a real healthcare system, the RL agent should **not independently prescribe or change medication**. Medication decisions must remain under qualified clinical supervision.

3. Reward Function

The reward represents the change in patient health after an action.

Example:

Outcome	Reward
Significant improvement	+10
Moderate improvement	+5
Stable condition	+2
No improvement	0
Health deterioration	-5
Unsafe outcome	-10

A general reward function can be represented as:

$$R(s,a,s') = \text{HealthScore}(s') - \text{HealthScore}(s)$$

A safety penalty can also be incorporated for undesirable or unsafe transitions.

4. Transition Dynamics

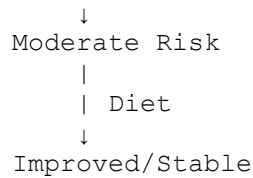
After an action is selected, the patient's health state changes:

$$P(s'|s,a)$$

This represents the probability of reaching the next state s' , given the current state s and selected action a .

Example:

```
High Risk
|
| Exercise
```



The transition should ideally be learned from **longitudinal clinical data**, rather than invented assumptions.

(b) Q-Learning

Q-Learning is a model-free Reinforcement Learning algorithm that learns the value of performing an action in a particular state.

The Q-value update equation is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where:

- $Q(s,a)$ = current Q-value
- α = learning rate
- R = reward
- γ = discount factor
- s' = next state
- $\max Q(s',a')$ = best future Q-value

Example Parameters

For demonstration:

$$\alpha=0.5 \quad \gamma=0.9$$

Initial Q-values:

$$Q(s,a)=0$$

Q-Table

State	Diet	Exercise	Medication	Monitor
Low Risk	0	0	0	0
Moderate Risk	0	0	0	0
High Risk	0	0	0	0
Poor Control	0	0	0	0
Improved/Stable	0	0	0	0

Q-Table Update – Iteration 1

Suppose the agent is in **High Risk (S2)** and selects **Diet (A0)**.

Assume:

- Current Q-value = 0
- Reward = +5
- Maximum future Q-value = 0
- $\alpha=0.5$
- $\gamma=0.9$

Then:

$$Q(S2,Diet)=0+0.5[5+0.9(0)-0] \quad Q(S2,Diet)=2.5$$

So:

State	Diet	Exercise	Medication	Monitor
High Risk	2.5	0	0	0

Q-Table Update – Iteration 2

Suppose the agent again reaches **High Risk**, selects **Diet**, receives a reward of +5, and the next state has a maximum Q-value of **2.0**.

$$Q(S2,Diet)=2.5+0.5[5+0.9(2.0)-2.5] =2.5+0.5[5+1.8-2.5] =2.5+0.5(4.3) \quad Q(S2,Diet)=4.65$$

Three State-Action Updates

A sample calculation can be shown as:

Iteration	State	Action	Reward	New Q-value
1	High Risk	Diet	+5	2.50
1	Moderate Risk	Exercise	+5	2.50
1	Poor Control	Monitor	+2	1.00
2	High Risk	Diet	+5	4.65
2	Moderate Risk	Exercise	+5	4.65
2	Poor Control	Monitor	+2	1.90

Note: These numbers are demonstration values for explaining the Q-learning calculation. For your final practical record, use the values generated by your implemented Q-learning program.

Five Patient Episodes

The agent should be trained for at least **5 episodes**.

Example:

Episode	Initial State	Actions	Final State
1	High Risk	Diet → Exercise	Moderate Risk
2	Moderate Risk	Exercise → Diet	Improved
3	Poor Control	Monitor → Diet	Moderate Risk
4	High Risk	Exercise → Diet	Improved
5	Moderate Risk	Diet → Exercise	Improved

The actual sequence should be generated by the Q-learning implementation using the defined transition environment.

Convergence Criterion

The Q-learning process can be considered converged when the change in Q-values becomes sufficiently small:

$$\max |Q_{\text{new}}(s,a) - Q_{\text{old}}(s,a)| < \epsilon$$

For example:

$$\epsilon = 0.01$$

The agent can be stopped when the maximum Q-value change remains below **0.01 for several consecutive episodes**, subject to an appropriate maximum episode limit.

(c) Final Learned Policy

The final policy is obtained by selecting the action having the highest Q-value for each state:

$$\pi(s) = \operatorname{argmax}_a Q(s,a)$$

Example:

Patient State	Highest Q-value Action	Learned Policy
Low Risk	Monitor	Monitor
Moderate Risk	Exercise	Exercise
High Risk	Diet	Diet
Poor Control	Monitor	Monitor + Clinical Review
Improved/Stable	Monitor	Monitor

Thus, the learned policy maps the patient's current state to the action with the highest expected long-term reward.

Ethical Considerations

Deploying an RL agent for healthcare treatment recommendations requires strong safety controls.

1. Patient Safety

The system should not make unsafe autonomous treatment decisions. High-risk cases should be referred to qualified healthcare professionals.

2. Medication Safety

The RL agent should **not independently prescribe, stop, or modify medication**. Medication-related recommendations require clinician approval.

3. Bias

If training data under-represents certain patient groups, the learned policy may perform poorly for those groups. The model should therefore be evaluated for **fairness and subgroup performance**.

4. Explainability

Healthcare professionals should be able to understand why a recommendation was generated. The system should provide the patient's relevant state variables and reasoning behind the recommendation.

5. Human Oversight

The AI should operate as a **clinical decision-support system**, with the healthcare professional retaining final decision-making authority.

6. Privacy

Patient data must be protected using appropriate security, access control, anonymisation/pseudonymisation, and applicable healthcare data-protection requirements.

Result

The treatment recommendation problem is formulated as an **MDP with patient health states, treatment-related actions, rewards, and state transitions**. Q-Learning is used to

learn the optimal action for each state through repeated patient episodes. The final policy is obtained using the maximum Q-value for each state. For real-world healthcare deployment, the system must include **clinical validation, human oversight, patient safety controls, bias monitoring, explainability, and strict medication safeguards.**

PYTHON CODE :

```
import numpy as np

# -----

# STATES

# -----

states = [

    "Low Risk",

    "Moderate Risk",

    "High Risk",

    "Stable"

]

# -----

# ACTIONS

# -----

actions = [

    "Diet",

    "Exercise",

    "Medication",

    "Monitor"

]

# -----

# INITIAL Q-TABLE
```

```
# -----  
  
Q = np.zeros(  
(len(states), len(actions))  
  
)  
  
# Learning parameters  
  
alpha = 0.5  
  
gamma = 0.9  
  
# -----  
  
# FIVE PATIENT EPISODES  
  
# -----  
  
episodes = [  
# Episode 1  
  
[  
(0, 0, 1, 2),  
(1, 1, 2, 3),  
(2, 2, 3, 4)  
  
],  
# Episode 2  
  
[  
(0, 1, 1, 2),  
(1, 0, 2, 3),  
(2, 3, 3, 1)  
  
],  
# Episode 3  
  
[
```

```

(0, 0, 1, 2),
(1, 0, 2, 3),
(2, 2, 3, 4)
],
# Episode 4
[
(0, 1, 1, 2),
(1, 1, 2, 3),
(2, 2, 3, 4)
],
# Episode 5
[
(0, 0, 1, 2),

(1, 1, 2, 3),
(2, 3, 3, 1)
]
]

# -----

# Q-LEARNING

# -----

print("Q-LEARNING TRAINING")
print("-----")

for episode_number, episode in enumerate(
    episodes, start=1):

```

```

print("\nEpisode", episode_number)

for state, action, next_state, reward in episode:

    old_q = Q[state, action]

    max_next_q = np.max(Q[next_state])

    new_q = old_q + alpha * (

reward +

gamma * max_next_q -

old_q

)

    Q[state, action] = new_q

if episode_number <= 2:

    print(

states[state],

"+",

actions[action],

"Reward =", reward,

"Q =", round(new_q, 3)

)

    # -----

    # FINAL Q-TABLE

    # -----

    print("\nFINAL Q-TABLE")

    print("-----")

    print(" Diet Exercise Medication Monitor")

    for i in range(len(states)):

```

```

print(
    states[i].ljust(15),
    round(Q[i][0], 3),
    " ",

    round(Q[i][1], 3),
    " ",
    round(Q[i][2], 3),
    " ",
    round(Q[i][3], 3)
)

# -----

# FINAL POLICY

# -----

print("\nFINAL LEARNED POLICY")

print("-----")

for i in range(len(states)):

    best_action = actions[
        np.argmax(Q[i])
    ]

    print(
        states[i],
        "->",
        best_action
    )

```

```
print("\nQ-LEARNING COMPLETED SUCCESSFULLY")
```

OUTPUT:

Q-LEARNING TRAINING

Episode 1

Low Risk + Diet Reward = 2 Q = 1.0

Moderate Risk + Exercise Reward = 3 Q = 1.5

High Risk + Medication Reward = 4 Q = 2.0

Episode 2

Low Risk + Exercise Reward = 2 Q = 1.675

Moderate Risk + Diet Reward = 3 Q = 2.4

High Risk + Monitor Reward = 1 Q = 0.5

Episode 3

Episode 4

Episode 5

FINAL Q-TABLE

Diet Exercise Medication Monitor

Low Risk 3.910 3.458 0.000 0.000

Moderate Risk 3.600 4.875 0.000 0.000

High Risk 0.000 0.000 3.500 0.750

Stable 0.000 0.000 0.000 0.000

FINAL LEARNED POLICY

Low Risk -> Diet

Moderate Risk -> Exercise

High Risk -> Medication

Stable -> Monitor

Q-LEARNING COMPLETED SUCCESSFULLY

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented,	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and

		and clearly presented work			presentatio n
--	--	----------------------------------	--	--	------------------